Univerzitet u Zenici
Politehnički fakultet

BILJEŠKE S PREDAVANJA · SEDMICA 2

Uvod u IaC: virtualne mašine kao kod

Principi Infrastructure as Code pristupa (deklarativnost, idempotentnost, state, drift) i prvi praktični alati: `Terraform`, `cloud-init` i `Packer`, na lokalnom KVM/QEMU/libvirt okruženju.

PREDMET	Računarstvo u oblaku
STUDIJ	II ciklus studija, III semestar, akademska godina 2026/27
NASTAVNIK	V. prof. dr. Đulaga Hadžić
ASISTENT	ass. Mr. Adin Jahić
TERMIN	petak, 23.10.2026, 17:15–20:15, Kabinet 3204
TEMA	Uvod u IaC: virtualne mašine kao kod
KOD	github.com/Dadyze/RuO-2026-27 (mape `s2/p1` do `s2/p4`)

OKVIR NASTAVNE JEDINICE

Ciljevi i način rada

U prvoj sedmici je kontejner razložen na mehanizme jezgra, a hipervizor (KVM/QEMU, libvirt) predstavljen kao drugi način izolacije. Ova sedmica odgovara na praktično pitanje: *kako se virtualne mašine prave na način koji je ponovljiv, provjerljiv i verzioniran?* Infrastruktura se opisuje tekstualnim datotekama u Git repozitoriju, a alat zadužen za izvršavanje dovodi stvarno stanje u skladu s opisom.

Svi primjeri rade **lokalno**, na vašem računaru: nema računa u oblaku i nema troškova. Resursi koje Terraform pravi su datoteke i KVM virtualne mašine, a ne AWS ili Azure instance. Gradivo ide od jednostavnog prema složenom: prvi primjer upravlja jednom datotekom i služi da se na najmanjem slučaju vide svi principi, a svaki sljedeći dodaje jedan sloj (cloud-init, prava VM, vlastita slika).

Nakon nastavne jedinice student je u stanju:

- objasniti razliku između imperativnog i deklarativnog pristupa i definisati idempotentnost, state i drift,
- objasniti ulogu KVM-a, QEMU-a i libvirt-a i kako Terraform preko providera upravlja virtualnim mašinama,
- napisati Terraform konfiguraciju i voditi tok `init → plan → apply → destroy`,
- napisati cloud-init `user-data` i objasniti kako VM dobija konfiguraciju pri prvom pokretanju,
- izgraditi "zlatnu" sliku Packerom i razlikovati ulogu Terraforma, Packera, cloud-inita i Ansible-a,
- pokrenuti sve primjere naredbom `make` i provjeriti da su idempotentni.

Kako se koristi ovaj materijal

Svaki isječak iz ovih bilješki je prava datoteka u repozitoriju, a pokreće se jednom naredbom. Dovoljno je kloniranje, jedno pokretanje skripte za instalaciju i zatim `make` po primjerima, redom.

```bash
# Isječak 0 · početak (u Linux/WSL2 home direktoriju)
cd ~
git clone https://github.com/Dadyze/RuO-2026-27.git
cd RuO-2026-27
./00-setup.sh      # jednom: instalira sve alate (objašnjeno u 4. poglavlju)
make check         # koji alati postoje
make help          # popis svih primjera
make s2-p1         # prvi primjer; zatim s2-p2, s2-p3, s2-p4 (ili: make all)
```

VAŽNO

Repozitorij klonirati u Linux fajl sistem (`~/`), ne u `/mnt/c/...`. Na Windows disku svi imaju pravo pisanja, pa Ansible (sedmica 3) ignoriše `ansible.cfg`.

---

1 · MOTIVACIJA

Zašto infrastruktura kao kod

Klasičan način rada je ručan: administrator se prijavi na server, instalira pakete, uredi konfiguraciju, otvori port. Taj pristup (u žargonu *ClickOps*) ima ponavljajuće probleme:

PROBLEM	POSLJEDICA
Ručne izmjene se ne bilježe	Nema odgovora ko je, kada i zašto promijenio konfiguraciju.
Server je "snowflake" (jedinstven)	Ako ga izgubimo, ne znamo ga ponovo napraviti. Test i produkcija se razilaze.
Ponavljanje je sporo i podložno grešci	Deset servera znači deset ručnih postupaka i deset blago različitih rezultata.
Nema pregleda prije izmjene	Greška se otkriva tek kad nešto prestane raditi.

IaC to rješava tako što se željeno stanje zapisuje kao kod, a kod se tretira kao svaki drugi softver: verzionira se u Gitu, pregleda (pull request), testira i izvršava automatski. Infrastruktura postaje **ponovljiva**, **provjerljiva** (historija je u Gitu) i **dokumentovana** (kod je dokumentacija koja ne može zastarjeti).

RADNA TEZA

Infrastruktura se ne *održava*, ona se *opisuje*. Ako server treba promijeniti, mijenja se opis i ponovo se izvršava, a ne uređuje se server.

Poznata metafora: server kao *kućni ljubimac* (pet) ima ime, njeguje se ručno i žali se kad pogine. Server kao *stoka* (cattle) ima broj, napravljen je iz koda i zamijeni se novim kad zakaže. IaC vodi prema stoci.

---

2 · PRINCIPI

Deklarativno, idempotentno, state, drift

2.1 Imperativno naspram deklarativnog

Imperativni pristup opisuje *korake* ("napravi ovo, pa ovo"). **Deklarativni** pristup opisuje *željeno stanje* ("treba postojati ovo"), a alat sam izračuna koji su koraci potrebni da se do njega dođe iz trenutnog stanja.

bash — Isječak 1 · imperativno (ilustracija)

```bash
#!/bin/bash
# kreiraj_vm.sh: drugo pokretanje puca jer VM već postoji
virt-install --name web1 --memory 1024 --vcpus 1 \
  --disk size=10 --import --os-variant ubuntu24.04 --noautoconsole
```

hcl — Isječak 2 · deklarativno (ilustracija)

```hcl
resource "libvirt_domain" "web1" {
  name   = "web1"
  memory = 1024
  vcpu   = 1
}
# "treba postojati VM web1 s 1 GiB memorije". Postoji li već, ne radi se ništa.
```

Skripta iz isječka 1 vrijedi za jedan početni slučaj: prvi put napravi VM, drugi put javlja grešku, a ako se memorija promijeni, ne pomaže ni to. Deklarativni zapis pokriva sva tri slučaja jer alat poredi željeno i stvarno stanje.

2.2 Idempotentnost

Operacija je **idempotentna** ako njeno ponovljeno izvršavanje daje isti rezultat kao jedno izvršavanje. U IaC-u to znači da se kod može bezbrižno pokrenuti više puta: ako je sistem već u željenom stanju, ništa se ne mijenja. To je preduslov za automatizaciju jer se isti kod izvršava pri svakoj izmjeni, a ne samo prvi put. Provjera je jednostavna: *drugo izvršavanje ne smije prijaviti nijednu promjenu*. Ova pravila važe i za sam repozitorij: svaki `make` cilj se može pokrenuti ponovo.

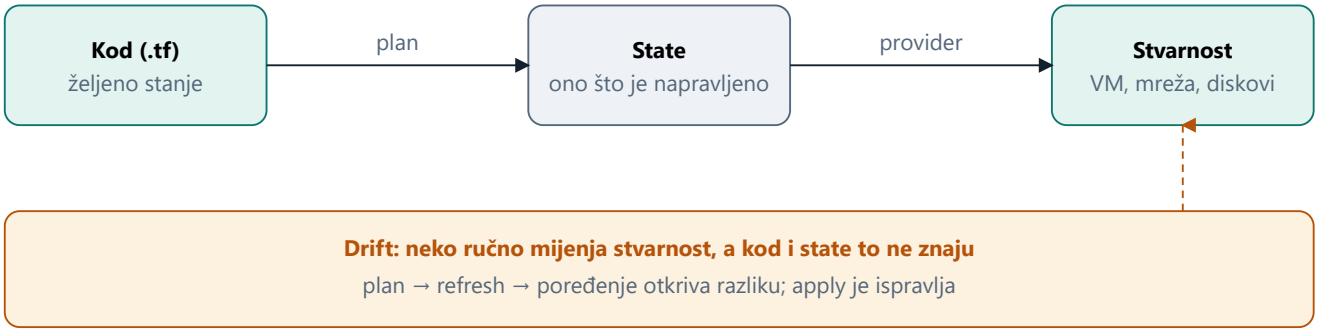
2.3 State

Alat mora znati šta je *on* napravio. Terraform to bilježi u datoteci **state** (`terraform.tfstate`) koja svaki resurs iz koda povezuje s konkretnim objektom u stvarnom svijetu (ID, atributi). Bez statea bi svaki `apply` morao pogađati šta je već napravljeno.

- State sadrži **osjetljive podatke** (lozinke, ključevi u čistom tekstu). Zato se nikad ne ubacuje u Git (`.gitignore`).
- Pri radu u timu state se čuva u **udaljenom backendu** s *zaključavanjem*, da dvije osobe ne izvršavaju `apply` istovremeno. U ovom kursu je state lokalna datoteka pored koda.
- Pregled: `terraform state list`, `terraform state show <resurs>`. Postojeći objekt se preuzima u state naredbom `terraform import`.

2.4 Drift

Drift je razlika između stanja zapisanog u kodu i stvarnog stanja, nastala izmjenom izvan alata (neko ručno promijeni konfiguraciju ili obriše resurs). Terraform drift otkriva pri svakom `plan` pozivu tako što prvo osvježi state (*refresh*), a zatim ga poredi s kodom. Samo otkrivanje, bez planiranja izmjena, radi `terraform plan -refresh-only`. Ispravlja se tako što `apply` vraća stvarno stanje na ono iz koda (ili se kod prilagodi ako je promjena bila namjerna).



Slika 1. Tri izvora istine: kod, state i stvarnost. Terraform stalno poredi sva tri, a drift nastaje kad se promijeni stvarnost mimo koda.

2.5 Promjenjiva i nepromjenjiva infrastruktura

PRISTUP	IDEJA	ALAT
Promjenjiva (*mutable*)	Server postoji i na njemu se mijenjaju paketi i konfiguracija. Dugim radom se razilazi od opisa.	Ansible (sedmica 3)
Nepromjenjiva (*immutable*)	Server se ne mijenja. Za svaku izmjenu pravi se nova slika i zamijeni se stari server.	Packer + Terraform

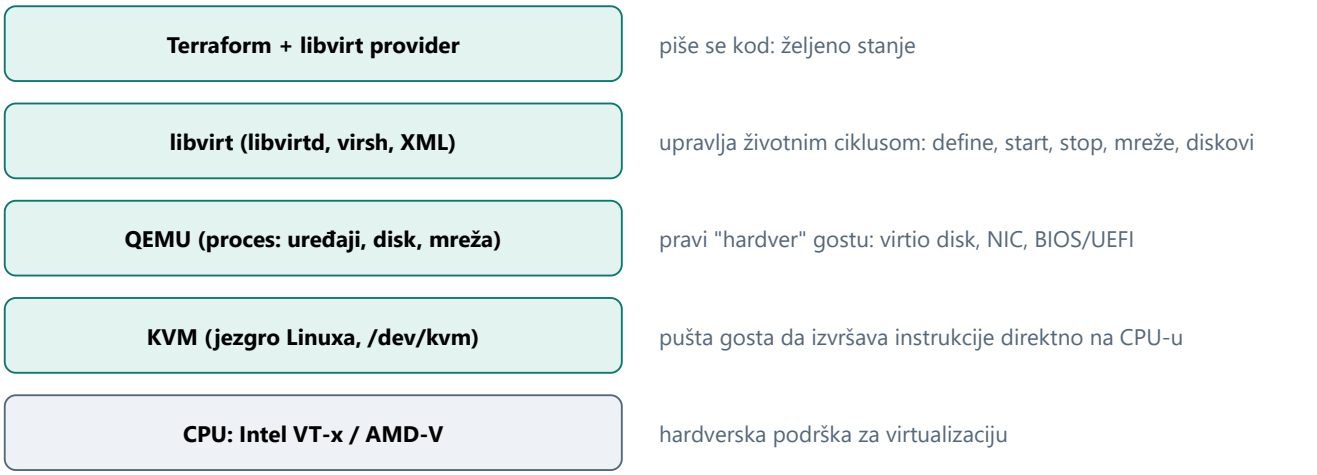
U praksi se kombinuju: slika sadrži ono što se rijetko mijenja, a ono što zavisi od okruženja (hostname, ključevi, adrese) dodaje se pri prvom pokretanju, što je posao cloud-inita.

---

3 · POZADINA

Virtualizacija na Linuxu: KVM, QEMU i libvirt

Primjeri 3 i 4 prave prave virtualne mašine, pa je potrebno znati od čega se okruženje sastoji. U prvoj sedmici je rečeno da hipervizor daje svakoj mašini vlastito jezgro. Na Linuxu tu ulogu ne igra jedan program, nego tri sloja koji se često miješaju:



Slika 2. Slojevi KVM virtualizacije. Terraform ne zna ništa o QEMU-u ili KVM-u: razgovara samo s libvirt-om.

POJAM	ŠTA JE I ZAŠTO JE TU
KVM	*Kernel-based Virtual Machine*: modul jezgra Linuxa (`kvm`, `kvm_intel` / `kvm_amd`) koji koristi procesorsku podršku za virtualizaciju. Izlaže uređaj `/dev/kvm`. Zahvaljujući njemu gostujući kod radi skoro brzinom domaćina. Sam po sebi ne pravi mašine.
QEMU	Program u korisničkom prostoru koji *pravi uređaje* gosta (disk, mrežnu karticu, BIOS/UEFI, konzolu). Svaka VM je jedan `qemu-system-x86_64` proces. Bez KVM-a QEMU sve emulira softverski (spor, ali radi); uz KVM ubrzava izvršavanje. Zato se kaže "KVM/QEMU".
libvirt	Jedinstven API i servis (`libvirtd`) za upravljanje hipervizorima (KVM/QEMU, Xen, LXC). VM opisuje XML "domenom"; alat `virsh` je njegov CLI. Terraform provider ga koristi umjesto da sam pokreće QEMU.
URI `qemu:///system`	Adresa libvirt servisa: sistemski servis (radi kao root, dijeli resurse). `qemu:///session` je varijanta po korisniku. Zato korisnik mora biti u grupi `libvirt`.
virtio	Paravirtualizovani uređaji: gost zna da je virtualan i koristi brz zajednički interfejs umjesto emulacije stvarne kartice. Cloud slike ga podrazumijevaju.
qcow2	*QEMU Copy-On-Write v2*: format diska koji raste tek kad se piše i podržava *backing file*. Disk VM-a je potomak bazne slike: čita iz nje, a piše samo svoje razlike. Zato nova VM zauzima malo prostora i nastaje za sekunde.
Cloud slika	Gotov, mali disk operativnog sistema (npr. `noble-server-cloudimg-amd64.img`) s ugrađenim cloud-initom. Ne dolazi s korisnicima ni lozinkama: sve se postavlja pri prvom pokretanju.
Pool i mreža `default`	Libvirt pool je direktorij za diskove (`/var/lib/libvirt/images`). Mreža `default` je NAT most `virbr0` (`192.168.122.0/24`) s DHCP-om (dnsmasq): VM izlazi na internet, ali je izvana nedostupna.

> POZADINA: WSL2
>
> WSL2 je i sam lagana virtualna mašina (Hyper-V). Da bi unutar nje radio KVM, potrebna je *ugnježđena virtualizacija*, što većina novijih računara podržava. Ako `/dev/kvm` ne postoji, VM primjeri ostaju na nivou `plan` / `validate`, što je i dovoljno za razumijevanje.

---

4 · SKRIPTA 00-SETUP.SH

Šta instalacijska skripta radi i zašto

Prva skripta u repozitoriju postavlja kompletno okruženje, da svi studenti rade u istom. Osim praktične koristi, ona je mali primjer iste ideje: **provjeri pa radi**. Svaki korak prvo ispituje stanje (je li alat već tu, postoji li ključ, je li korisnik u grupi) i mijenja samo ono što nedostaje, pa se skripta može pokrenuti više puta (idempotentnost, 2.2).

KORAK	ŠTA RADI	ZAŠTO
1. Okruženje	Provjerava Linux, `apt`, WSL2; ako treba, upisuje `systemd=true` u `/etc/wsl.conf`.	libvirt je servis koji traži `systemd`. Bez provjere bi kasnije greške bile nejasne.
2. Osnovni paketi	`git`, `make`, `curl`, `python3-yaml`, `ansible`.	Alati kojima se pokreću primjeri; `make` je ulaz u sve.
3. HashiCorp repozitorij	Dodaje potpisani apt repozitorij (`signed-by` ključ), instalira `terraform` i `packer`.	Ubuntu repozitorij nema ove alate. Potpisan repozitorij znači provjeru autentičnosti paketa; ne koristi se zastarjeli `apt-key`.
4. KVM i libvirt	`qemu-system-x86`, `libvirt-daemon-system`, `virtinst`, `genisoimage`; dodaje korisnika u grupe `libvirt` i `kvm`; pokreće `libvirtd`; pravi pool i mrežu `default`.	Slojevi sa slike 2. Grupe daju pravo na `/dev/kvm` i libvirt socket bez `sudo`. `genisoimage` pravi `cidata` ISO za cloud-init. Primjeri očekuju resurse imenom `default`.
5. Docker	Provjerava da radi; s `INSTALL_DOCKER=1` ga instalira.	Potreban za sedmicu 3. U WSL2 je jednostavniji Docker Desktop.
6. kubectl i kind	Preuzima binarne datoteke u `/usr/local/bin` ako ih nema.	Za Kubernetes primjer (sedmica 3); `kind` pravi klaster u Docker kontejnerima.
7. SSH ključ	Pravi `~/.ssh/id_ed25519` ako ne postoji.	Javni ključ ulazi u VM preko cloud-inita, pa se prijavljujemo bez lozinke.
8. Provjera	Ispisuje koji su alati prisutni i njihove verzije.	Brza potvrda prije prvog primjera.

bash — Isječak 3 · opcije skripte

```
./00-setup.sh                  # sve
SKIP_LIBVIRT=1 ./00-setup.sh   # bez KVM-a (ako nema /dev/kvm)
SKIP_K8S=1 ./00-setup.sh       # bez kubectl i kind
INSTALL_DOCKER=1 ./00-setup.sh # instalira i Docker Engine
```

Nakon prvog pokretanja treba se odjaviti i ponovo prijaviti (u WSL2: `wsl --shutdown`) jer članstvo u novim grupama počinje važiti tek u novoj sesiji.

---

5 · TERRAFORM

Alat i tok rada

Terraform (HashiCorp, 2014, napisan u jeziku Go) je deklarativni alat koji infrastrukturu opisuje u jeziku **HCL** (*HashiCorp Configuration Language*). Sam alat ne zna ništa o konkretnim sistemima: to znaju **provideri**, dodaci koje `terraform init` preuzima iz javnog registra. Postoje provideri za AWS, Azure, Docker, DNS, libvirt, pa čak i za lokalne datoteke. Isti tok rada vrijedi za sve, pa znanje stečeno na lokalnom libvirt-u prelazi na oblak.

Građevni blokovi HCL-a

BLOK	ULOGA
`terraform { required_providers }`	Koji provideri i koje verzije se koriste. Verzija se ograničava (`~> 0.8.3`) jer nove verzije mogu promijeniti sintaksu.
`provider "x" { }`	Postavke providera (npr. adresa libvirt servisa).
`resource "tip" "ime" { }`	Objekt kojim se upravlja. Referencira se kao `tip.ime.atribut`.
`data "tip" "ime" { }`	Samo čitanje postojećeg objekta.
`variable "ime" { }`	Ulazni parametar (`-var`, `terraform.tfvars`).
`output "ime" { }`	Vrijednost koja se ispisuje nakon `apply` (npr. IP adresa).

Redoslijed izvršavanja se *ne* piše ručno. Terraform iz referenci među resursima (disk koji koristi bazni volumen) gradi graf zavisnosti, pa sam odredi poredak i paralelizam.

Tok rada

NAREDBA	ŠTA RADI
`terraform init`	Preuzima providere u `.terraform/` i zaključava verzije u `.terraform.lock.hcl` (ova datoteka ide u Git).
`terraform validate`	Provjerava sintaksu i reference.
`terraform plan`	Osvježava state, poredi s kodom i ispisuje *šta bi se uradilo*. Ništa ne mijenja. Oznake: `+` kreiranje, `~` izmjena, `-` brisanje, `-/+` zamjena.
`terraform apply`	Izvršava plan i upisuje rezultat u state.
`terraform destroy`	Uklanja sve resurse iz statea.

> **NAVIKA: PLAN PRIJE APPLY**
> `plan` se čita kao *pull request* za infrastrukturu i ne mijenja ništa. Zato je u repozitoriju za VM primjere osnovni cilj samo `plan`, a `apply` je opcionalan. Posebno paziti na `-/+` i `-` (brisanje podataka).

---

6 · PRIMJER 1 · S2/P1

Terraform bez oblaka: idempotentnost, state i drift

> **POZADINA PRIMJERA**
> Provider `hashicorp/local` upravlja običnom datotekom. Resurs je trivijalan, ali se na njemu vide sva četiri principa iz 2. poglavlja i ne treba ni virtualizacija ni račun u oblaku. Zato je to prvi primjer.

hcl — Isječak 4 · s2/p1/main.tf

```hcl
terraform {
  required_providers {
    local = { source = "hashicorp/local", version = "~> 2.5" }
  }
}

variable "ime" {
  type    = string
  default = "RuO"
}

resource "local_file" "pozdrav" {
  filename        = "${path.module}/pozdrav.txt"
  content         = "Zdravo, ${var.ime}!\n"
  file_permission = "0644"
}

output "putanja" { value = local_file.pozdrav.filename }
```

bash — Isječak 5 · pokretanje

```bash
make s2-p1          # init, plan, apply, ponovni apply, promjena varijable, drift, ispravak
make s2-p1-clean    # destroy i brisanje statea
```

Cilj `make s2-p1` vodi kroz šest koraka, svaki s naslovom, a naredbe se ispisuju prije izvršavanja:

KORAK	ŠTA SE VIDI
1. `init` i `plan`	`Plan: 1 to add`: datoteka još ne postoji.
2. `apply`	`Apply complete! Resources: 1 added`: datoteka je napravljena, a u `terraform.tfstate` je upisan resurs.
3. `apply` ponovo	`Resources: 0 added, 0 changed`: **idempotentnost**.
4. promjena varijable	Plan prikazuje `-/+` i oznaku `# forces replacement`, prije bilo kakve izmjene.
5. ručna izmjena datoteke	`plan -refresh-only` otkriva **drift** (datoteka je izmijenjena izvan Terraforma).
6. `apply`	Stvarno stanje se vraća na ono iz koda: `Zdravo, RuO!`

izlaz koraka 3 i 6 (skraćeno)

```
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.     # korak 3
local_file.pozdrav
...
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.     # korak 6
Zdravo, RuO!
```

> **DETALJ OVOG PROVIDERA**
>
> Provider `local` identitet resursa veže za hash sadržaja, pa izmijenjenu datoteku prijavljuje kao "obrisanu" i ponovo je kreira. Kod drugih providera (npr. memorija VM-a) isti drift se prikazuje kao `~ update in-place`. Princip je isti: razlika se uoči u `plan`, a `apply` je ispravlja.

7 · CLOUD-INIT · PRIMJER 2 · S2/P2

Konfiguracija pri prvom pokretanju

POZADINA

cloud-init je standardni alat u slikama oblaka (razvio ga je Canonical za Ubuntu na AWS-u, danas ga koriste skoro sve cloud slike). Pri *prvom* pokretanju čita konfiguraciju iz izvora podataka (*datasource*) i primjenjuje je: hostname, korisnici i SSH ključevi, paketi, datoteke, naredbe. Na AWS-u se konfiguracija čita sa metadata servisa. U lokalnom KVM/libvirt okruženju koristi se datasource *NoCloud*: mala ISO slika s oznakom `cidata` koja se priključi VM-u kao CD. Zato je u skripti instaliran `genisoimage`: Terraform provider i Packer njime prave tu ISO sliku.

DATOTEKA	SADRŽAJ
`user-data`	Šta se radi na mašini. Počinje redom `#cloud-config`.
`meta-data`	Identitet instance (`instance-id`). Kad se promijeni, cloud-init to smatra novom mašinom i ponovo se izvršava.
`network-config`	Opcionalna mrežna konfiguracija.

yaml — Isječak 6 · s2/p2/user-data

```yaml
#cloud-config
hostname: web1
users:
  - name: student
    groups: [sudo]
    sudo: ALL=(ALL) NOPASSWD:ALL
    shell: /bin/bash
    lock_passwd: true                # prijava samo ključem
    ssh_authorized_keys:
      - ssh-ed25519 AAAA...vas_javni_kljuc student@racunar

package_update: true
packages: [nginx, curl]

write_files:
  - path: /var/www/html/index.html
    permissions: "0644"
    content: |
      <h1>web1: konfigurisano cloud-initom</h1>

runcmd:
  - systemctl enable --now nginx
```

bash — Isječak 7 · provjera

```bash
make s2-p2    # prvi red je #cloud-config; cloud-init schema (ili YAML provjera ako cloud-init nije instaliran)
```

GREŠKA KOJA SE ČESTO PRAVI

YAML je osjetljiv na uvlačenje, a prvi red mora biti tačno `#cloud-config`. Pogrešan user-data cloud-init *ne prijavi kao grešku pri pokretanju*: mašina se podigne bez korisnika, pa se ne možemo prijaviti. Dnevnik je u `/var/log/cloud-init-output.log`, a stanje daje `cloud-init status --long`.

---

8 · PRIMJER 3 · S2/P3

Virtualna mašina u libvirt-u iz Terraforma

POZADINA PRIMJERA

U 1. sedmici su VM pravljene ručno preko `virsh`. Sada Terraform preko providera `dmacvicar/libvirt` razgovara s libvirt servisom (slika 2) i pravi četiri resursa: *bazni volumen* (preuzeta Ubuntu cloud slika), *disk VM-a* (qcow2 potomak bazne slike), *cloud-init ISO* i *domenu* (samu VM). Isječak 6 postaje šablon `cloud-init.tftpl` jer se hostname i ključ ubacuju Terraformovom interpolacijom `${...}`.

hcl — Isječak 8 · s2/p3/main.tf (skraćeno)

```hcl
provider "libvirt" { uri = "qemu:///system" }

# bazna slika se preuzima jednom; disk VM-a je njen "copy-on-write" potomak
resource "libvirt_volume" "base" {
  name   = "ubuntu-noble-base.qcow2"
  pool   = "default"
  format = "qcow2"
  source = "https://cloud-images.ubuntu.com/noble/current/noble-server-cloudimg-amd64.img"
}

resource "libvirt_volume" "disk" {
  name           = "${var.vm_name}.qcow2"
  pool           = "default"
  base_volume_id = libvirt_volume.base.id
  size           = 10 * 1024 * 1024 * 1024
}

resource "libvirt_cloudinit_disk" "init" {
  name      = "${var.vm_name}-init.iso"
  pool      = "default"
  user_data = templatefile("${path.module}/cloud-init.tftpl", {
    hostname = var.vm_name
    ssh_key  = trimspace(file(pathexpand("~/.ssh/id_ed25519.pub")))
  })
}

resource "libvirt_domain" "vm" {
  name      = var.vm_name
  memory    = 1024
  vcpu      = 1
  cloudinit = libvirt_cloudinit_disk.init.id
  disk { volume_id = libvirt_volume.disk.id }
  network_interface {
    network_name   = "default"
    wait_for_lease = true
  }
}
```

```
output "ip" { value = libvirt_domain.vm.network_interface[0].addresses[0] }
```

bash — Isječak 9 · pokretanje

```
make s2-p3          # init, validate, PLAN (ništa se ne kreira)
make s2-p3-apply    # opcionalno: stvarno kreira VM (traži KVM)
make s2-p3-clean    # uklanja VM i state
```

izlaz plana (skraćeno)

```
  # libvirt_volume.base will be created
  # libvirt_volume.disk will be created
  # libvirt_cloudinit_disk.init will be created
  # libvirt_domain.vm will be created

Plan: 4 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + ip = (known after apply)
```

Plan se čita kao pregled: četiri resursa se kreiraju, nijedan se ne mijenja ni briše. Vrijednost `ip` je `(known after apply)` jer adresa ne postoji dok VM ne dobije zakup od DHCP-a: plan ne može znati ono što još ne postoji. U ovom primjeru je i vidljiva prednost qcow2 potomka: druga VM je promjena jedne varijable (`vm_name`), a zauzima samo prostor svojih razlika.

ČESTE PREPREKE

- `Permission denied` pri otvaranju diska: AppArmor nad `/var/lib/libvirt/images`. Za lab: `security_driver = "none"` u `/etc/libvirt/qemu.conf` i `sudo systemctl restart libvirtd`.
- Nema `/dev/kvm`: ostati na `plan`; za stvarno pokretanje bi u domeni trebalo `type = "qemu"` (emulacija, sporo).
- Provider je zakucan na seriju 0.8.x jer novija serija ima drugačiju sintaksu resursa.

---

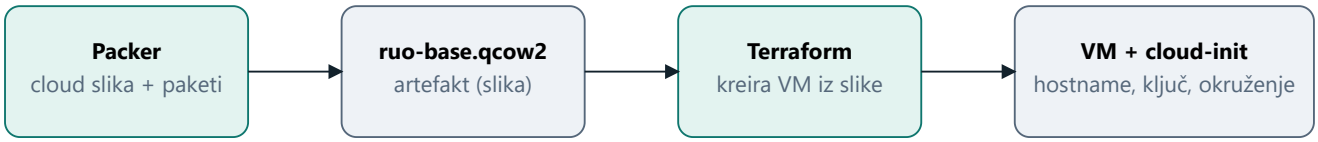
9 · PACKER · PRIMJER 4 · S2/P4

Slika kao artefakt

POZADINA

Packer (HashiCorp) pravi slike mašina iz koda. Radi tako što pokrene privremenu VM iz osnovne slike (builder, ovdje `qemu` plugin), izvrši provisionere (shell skripta ili Ansible), ugasi mašinu i sačuva disk kao *artefakt*. Za iste slike se koriste termini "golden image" ili "pečena" slika (*baked image*).

U primjeru 3 mašina pri svakom pokretanju preuzima i instalira nginx. To je sporo i zavisi od mreže. Packer taj posao premješta u fazu *izgradnje slike*: jednom se instaliraju paketi, a Terraform ubuduće pokreće mašine iz gotove slike. To je nepromjenjiva infrastruktura u praksi.



Slika 3. Podjela posla: Packer pravi sliku (sporo, jednom), Terraform pravi mašine iz nje (brzo, puno puta), a cloud-init ih personalizuje.

hcl — Isječak 10 · s2/p4/ubuntu.pkr.hcl (skraćeno)

```hcl
source "qemu" "ubuntu" {
  iso_url          = "https://cloud-images.ubuntu.com/noble/current/noble-server-cloudimg-amd64.img"
  iso_checksum     = "file:https://cloud-images.ubuntu.com/noble/current/SHA256SUMS"
  disk_image       = true          # ulaz je gotov disk, ne instalacijski ISO
  format           = "qcow2"
  accelerator      = "kvm"         # bez KVM-a: "tcg" (sporo)
  cd_files         = ["./seed/user-data", "./seed/meta-data"]   # privremeni cidata ISO
  cd_label         = "cidata"
  ssh_username     = "ubuntu"
  ssh_password     = "packer"
  output_directory = "output"
  vm_name          = "ruo-base.qcow2"
}

build {
  sources = ["source.qemu.ubuntu"]
  provisioner "shell" {
    inline = [
      "sudo cloud-init status --wait",
      "sudo apt-get update",
      "sudo DEBIAN_FRONTEND=noninteractive apt-get install -y nginx qemu-guest-agent",
      "sudo systemctl enable nginx",
      "sudo passwd -l ubuntu",                  # zaključava lozinku korištenu samo pri izgradnji
      "sudo cloud-init clean --logs --seed",    # cloud-init se ponovo izvršava pri prvom pokretanju
    ]
  }
}
```

bash — Isječak 11 · pokretanje

```bash
make s2-p4          # packer init i validate (bez pokretanja VM-a)
make s2-p4-build    # opcionalno: gradi sliku (~4 min, ~640 MB, traži KVM)
make s2-p4-vm       # plan VM-a iz izgrađene slike (bazni volumen je sada lokalna datoteka)
make s2-p4-clean
```

Posljednja dva reda provisionera su presudna. `passwd -l` zaključava lozinku korištenu samo pri izgradnji, a `cloud-init clean` briše tragove prvog pokretanja, pa se cloud-init *ponovo izvršava* pri pokretanju mašine iz slike i primjenjuje nove korisnike i ključeve. Bez toga bi mašine iz slike bile nepristupačne. U `s2/p4/vm/main.tf` bazni volumen više ne dolazi s interneta nego iz `../output/ruo-base.qcow2`, a nginx se ne instalira pri pokretanju. Kad se zahtjev promijeni (nova verzija nginx-a), mijenja se jedna linija u Packeru, pravi se nova slika, a Terraform zamijeni mašine: mašine se ne "krpe".

10 · PREGLED

Ko šta radi i redoslijed vježbi

ALAT	FAZA	ODGOVARA NA PITANJE	PRIMJER
Packer	izgradnja slike	Šta je u slici (OS, paketi)?	s2/p4
Terraform	kreiranje resursa	Koje mašine, diskovi i mreže postoje?	s2/p1, p3
cloud-init	prvo pokretanje	Ko je ova mašina (hostname, korisnici, ključevi)?	s2/p2, p3
Ansible	konfiguracija u radu	Kako je mašina podešena i kako se to održava?	sedmica 3

Sekvencijalno pokretanje i idempotentnost

Primjeri se pokreću `make` naredbom, redom. Svaki cilj je **idempotentan**: može se pokrenuti ponovo i završava u istom stanju, pa se bez straha mogu ponavljati tokom nastave.

#	NAREDBA	ŠTA POKAZUJE	PRI PONOVNOM POKRETANJU
1	`make s2-p1`	state, idempotentnost, drift	isti tok; datoteka se vraća na stanje iz koda
2	`make s2-p2`	validacija cloud-init user-data	samo provjera, bez izmjena
3	`make s2-p3`	plan VM-a iz Terraforma	novi plan, ništa se ne kreira
4	`make s2-p4`	validacija Packer koda	samo provjera
·	`make all`	sve gore (i sedmica 3) redom	isti rezultat
·	`make s2-p3-apply`, `s2-p4-build`	opcionalno stvarno kreiranje (KVM)	drugi `apply`: `0 added, 0 changed`

Rad u Git repozitoriju

- U repozitorij idu `*.tf`, `*.pkr.hcl`, `cloud-init.tftpl` i `.terraform.lock.hcl`. Ne idu `.terraform/`, `*.tfstate`, slike (`*.qcow2`) i tajne.
- Svaka izmjena infrastrukture je commit, a prije spajanja se pregleda izlaz `plan`.
- Struktura iz repozitorija je i oblik projektnog zadatka (25 bodova): kod, dokumentacija arhitekture i mogućnost da ga drugi student pokrene jednom naredbom.

Pitanja za provjeru

PITANJE	ODGOVOR
Zašto se `terraform.tfstate` ne stavlja u Git?	Sadrži tajne u čistom tekstu, a pri radu u timu uzrokuje sukobe. Čuva se u zaključanom udaljenom backendu.
Čemu služi KVM, a čemu QEMU?	KVM daje gostu izvršavanje na CPU-u (jezgro Linuxa). QEMU pravi uređaje gosta. libvirt njima upravlja.
Šta pokazuje `plan` kad je neko ručno promijenio resurs?	Drift: razliku između statea nakon osvježavanja i koda. `apply` stvarno stanje vraća na ono iz koda.
Zašto je `cloud-init clean` u Packer skripti?	Da se cloud-init izvrši ponovo pri prvom pokretanju svake mašine iz slike.
Kada slika, a kada cloud-init?	Slika: ono što je isto za sve mašine. cloud-init: ono što je specifično za instancu.

Zadaci za samostalni rad

1. U `s2/p1` dodati drugu datoteku pomoću `for_each` nad listom imena i pratiti kako `plan` reaguje kad se element ukloni iz liste.
2. U `s2/p3` napraviti dvije mašine (`web1`, `web2`) pomoću `for_each` i dodati izlaz s obje adrese. Predati izlaz `terraform plan` (očekuje se `Plan: 7 to add`) bez izvršavanja `apply`.
3. Proširiti Packer sliku jednim paketom, promijeniti verziju u `/etc/ruo-image-version` i opisati šta bi se desilo s mašinama.

> **SLJEDEĆA SEDMICA**
> Mašine su sada kreirane iz koda, ali se još ručno "dorađuju". Sedmica 3 uvodi Ansible (konfiguracija u radu), povezivanje s Terraformom i kontejnere kao kod (Dockerfile, Compose, Kubernetes manifesti).